数据与算法

CH03 线性表

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

本节内容

- 线性表的定义及ADT
- 顺序表
- 单向链表
- 双向链表

线性表的定义与ADT

- **很多自然或人工数据可按某种‘顺序’线性排列**
  - {‘A’，‘B’，‘C’，‘D’，…}
  - {‘氢’，‘氦’，‘锂’，‘铍’，‘硼’，…}
  - {‘小学’，‘初中’，‘高中’，‘大学’，…}

月份	缴费日期	缴费额
1	2005.1.20	88.00
2	2005.2.20	72.80
3	2005.3.20	102.20
…	…	…

线性表的定义与ADT

- **这些数据元素具有‘相同’的性质；**
- **数据元素之间存在着明确的顺序关系：**
  - 存在唯一的‘第一个’数据元素，无前驱(前件)
  - 存在唯一的‘最后一个’数据元素，无后继(后件)
  - 除了‘第一个’数据元素外，其他数据元素均有且只有一个直接前驱
  - 除了‘最后一个’数据元素外，其他数据元素均有且只有一个直接后继

线性表的定义与ADT

- **线性表(linear list)：具有相同性质的数据元素顺序排列形成的有限序列.**

$$B = (D,R)$$
$$D = \{a_i \mid i=0,1,2,\ldots,n-1\}$$
$$R = \{< a_i, a_{i+1} > \mid i=0,1,2,\ldots,n-2\}$$

或简单表示

线性表长度为n，n=0则为空表

$$B = (a_0, a_1, a_2, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_{n-1})$$

a_0：第一个数据元素

a_{i-1}：a_i的直接前驱

a_i：第i个数据元素

a_{i+1}：a_i的直接后继

a_{n-1}：最后一个数据元素

线性表的定义与ADT

- **线性表的ADT**
  - 数据元素操作：查找，插入，删除，遍历
  - 属性操作：属性读取
  - 关系访问：访问表中特定关系的数据元素

```
ADT LinearList{
    数据对象：D = {a_i | i ∈ ElemSet, i = 0,1,...,n − 1}
    数据关系：R = {< a_i, a_{i+1} > | a_i ∈ D, i = 0,1,...,n − 2}
    基本操作：

    InitList(&L)
        操作结果：构造一个空线性表.

    DestroyList(&L)
        初始条件：线性表L已存在.
        操作结果：销毁线性表L.
```

线性表的定义与ADT

……

ListEmpty(L)

初始条件：线性表L已存在.

操作结果：若L空，返回1，否则返回0.

ListLength(L)

初始条件：线性表L已存在.

操作结果：返回L的长度n.

GetElem(L,i,&e)

初始条件：线性表L已存在,$0 \le i \le n-1$.

操作结果：用e返回L的第i个数据元素值.

SetElem(&L, i, e)

初始条件：线性表L存在，$0 \le i \le n-1$.

操作结果：将L的第i个位置数据元素赋值为e.

……

线性表的定义与ADT

……

PriorElem(L, cur_e, &pre_e)

初始条件：线性表L已存在.

操作结果：若cur_e是L的数据元素且非第一个，则以pre_e返回其前驱数据元素值；否则操作失败pre_e无效.

NextElem(L, cur_e, &suc_e)

初始条件：线性表L已存在.

操作结果：若cur_e是L的数据元素且不是最后一个，则以suc_e返回其后继数据元素值；否则操作失败suc_e无效.

FindElem(L, e, equal())

初始条件：线性表L存在，equal()判断数据元素是否相等.

操作结果：返回L中首个与e满足equal()判定的数据元素的位置，找不到则返回 -1.

Insert(&L, i, e)

初始条件：线性表L已存在，0≤i≤n.

操作结果：在L的第i个位置插入新的数据元素e，L长度加1.

……

线性表的定义与ADT

……

Delete(&L, i, &e)

初始条件：线性表L已存在，0≤i≤n-1.

操作结果：删除L的第i个元素，用e返回其值，L长度减1.

Traverse(L, visit())

初始条件：线性表L已存在.

操作结果：依次对L的每个数据元素调用visit().

ClearList(&L)

初始条件：线性表L存在.

操作结果：将线性表重置为空.

}ADT LinearList

线性表的定义与ADT

- **求以线性表la，lb表示的两个集合的差集**

```
void diff(LinearList la, LinearList lb, LinearList &lc){
    ClearList(lc);                          //清空lc表
    for(i=0;i<ListLength(la);i++){
        GetElem(la, i, e);                  //取出la表中元素
        if(FindElem(lb,e,equal)==-1){       //查找lb表
            Insert(lc,ListLength(lc),e);    //插入操作
        }
    }
}                                           //lc=la-lb
```

线性表的定义与ADT

- 已知线性表la元素的数据类型为int，调整该线性表使得所有奇数元素的位置都小于任何偶数元素的位置.

```
void adjust(&la){
    left=0; right=ListLength(la)-1;
    while(left<right){
        GetElem(la,left,e1);
        GetElem(la,right,e2);
        if(e1%2==1) left++;                //e1为奇数
        else if(e2%2==0) right--;          //e2为偶数
        else{                              //交换e1,e2位置
            SetElem(la,left++,e2);
            SetElem(la,right--,e1);
        }
    }
}
```

(5,8,11,6,7,4,3) → (5,3,11,7,6,4,8)

本节内容

- 线性表的定义及ADT
- 顺序表
- 单向链表
- 双向链表

顺序表

- **线性表的顺序存储方式-顺序表(sequential list)**
  - 数据元素依次顺序存储在一组连续的存储单元中
  - 数据元素的物理存放顺序与其逻辑顺序一致

序号	内存中元素	存储地址
1	a_0	$addr(a_0)$
2	a_1	$addr(a_0)+k$
3	a_2	$addr(a_0)+2k$
…	…	…
n-1	a_{n-2}	$addr(a_0)+(n-2)k$
n	a_{n-1}	$addr(a_0)+(n-1)k$

基地址

  - C++中可采用数组(静态或动态)方式实现

顺序表

- **顺序表的基本操作**
  - InitList/DestroyList: 分配/释放数组，表长置0
  - ListEmpty: 返回表长是否为0
  - ClearList：表长置0
  - ListLength: 返回当前表长
  - GetElem/SetElem：返回/修改给定位置元素值
  - FindElem： 依次判断数据元素是否符合查找目标
  - PriorElem/NextElem: 找到当前元素，返回其前/后位置元素
  - Traverse： 依次访问数据元素
  - Insert/Delete：关键的操作，可能影响多个元素...

顺序表

- **顺序表的插入操作：在已有的顺序表第i个位置插入一个新元素；表长加1，i之后的元素均需后移1位；**
  - 插入前：$(a_0, a_1, a_2 \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$
  - 插入后：$(a_0, a_1, a_2 \dots, a_{i-1}, \boldsymbol{a}^*, a_i, , a_{i+1}, \dots, a_{n-1})$
- **算法分析**
  - 第i个元素开始到表尾共$n-i$个元素后移一位
  - 在第i个位置插入元素的先验概率为p_i
  - 移动元素次数期望：$E_{ins} = \sum_{i=0}^{n} p_i(n-i)$
  - 空间需求：一个数据元素辅助单元

顺序表

- **顺序表的删除操作：删除顺序表第i个位置元素**

  表长减1，i之后的元素均需前移1位；

  - 删除前：$(a_0, a_1, a_2 \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$
  - 删除后：$(a_0, a_1, a_2 \dots, a_{i-1}, a_{i+1}, \dots, a_{n-1})$

- **算法分析**

  - 第$i+1$个元素开始到表尾共$n-i-1$个元素前移一位
  - 删除第i个位置元素的先验概率为q_i
  - 移动元素次数期望：$E_{del} = \sum_{i=0}^{n-1} q_i(n-i-1)$；
  - 空间需求：一个数据元素辅助单元.

顺序表

- **假设插入/删除的概率符合均匀分布：**
- $\boldsymbol{p_i} = \frac{1}{n+1}$
- $\boldsymbol{q_i} = \frac{\mathbf{1}}{\boldsymbol{n}}$
- **则有：**

$$E_{is} = \sum_{i=0}^{n} (n-i) \Big/ (n+1) = \frac{n}{2}$$

$$E_{de} = \sum_{i=0}^{n-1} (n-i-1)/n = \frac{n-1}{2}$$

线性表插入/删除一个元素的平均时间复杂度约为其长度的一半，或者O(n)

顺序表

- C++实现

```
template <class ElemType>
class SeqList{
    private:
        ElemType* list;
        int len;
        int maxLen;
    public:
        SeqList(int m) { maxLen=m; list=new ElemType[m];
                len=0; }            //InitList
        ~SeqList() { delete[] list; }  //DestroyList
        bool ListEmpty() { return len==0; }
        ClearList() { len=0; }
        int ListLength() { return len; }
        bool GetElem(int pos, ElemType &e);
        bool SetElem(int pos, ElemType e);
        bool FindElem(int &pos, ElemType e);
        bool PriorElem(ElemType cur_e, ElemType &pre_e);
        bool NextElem(ElemType cur_e, ElemType &suc_e);
        bool Insert(int pos, ElemType e);
        bool Delete(int pos, ElemType &e);
        Traverse(void (*visit)(ElemType &e));
}
```

```
template <class ElemType>          //取pos位置的数据元素
bool SeqList<ElemType>::GetElem(int pos, ElemType &e){
    if(pos<0||pos>=len) return false; //out of range
    e = list[pos]; return true;
}

template <class ElemType>          //设置pos位置的数据元素
bool SeqList<ElemType>::SetElem(int pos, ElemType e){
    if(pos<0||pos>=len) return false; //out of range
    list[pos] = e; return true;
}

template <class ElemType>          //查找值为e的首个数据元素
bool SeqList<ElemType>::FindElem(int& pos, ElemType e){
    pos = 0;
    while(pos<len && list[pos]!=e) pos++;
    return pos<len;                    //found or not
}
```

```
template <class ElemType>          //当前元素的前驱
bool SeqList<ElemType>::PriorElem(ElemType cur_e,
ElemType &pre_e){
    int pos;
    if(FindElem(pos, cur_e)){
        if(pos>0){                 //not the first elem
            pre_e=list[pos-1];
            return true; }
    }
    return false; }

template <class ElemType>          //当前元素的后继
bool SeqList<ElemType>::NextElem(ElemType cur_e,
ElemType &pre_e){
    int pos;
    if(FindElem(pos, cur_e)){
        if(pos<len-1){             //not the last elem
            pre_e=list[pos+1];
            return true; }
    }
    return false; }
```

```
template <class ElemType>        //在pos位置插入数据元素
bool SeqList<ElemType>::Insert(int pos, ElemType e){
    if(pos<0||pos>len) return false; //out of range
    if(len+1>maxLen) return false;   //overflow
    for(int i=len;i>pos;i--)
        list[i] = list[i-1];         //元素后移
    list[pos] = e;    len++;         //插入，表长加1
    return true;
}
```

memmove

```
template <class ElemType>        //删除pos位置的数据元素
bool SeqList<ElemType>::Delete(int pos, ElemType &e){
    if(pos<0||pos>len-1) return false; //out of range
    e = list[pos]; len--;              //删除，表长减1
    for(int i=pos;i<len;i++)
        list[i] = list[i+1];           //元素前移
    return true;
}
```

顺序表

```
template <class ElemType>     //遍历表中全部数据元素
bool SeqList<ElemType>::Traverse(bool (*visit)(ElemType
& e)){
    int i=0;
    while( i<len && visit(list[i]) i++;
    return i==len;            //是否遍历所有元素
}
```

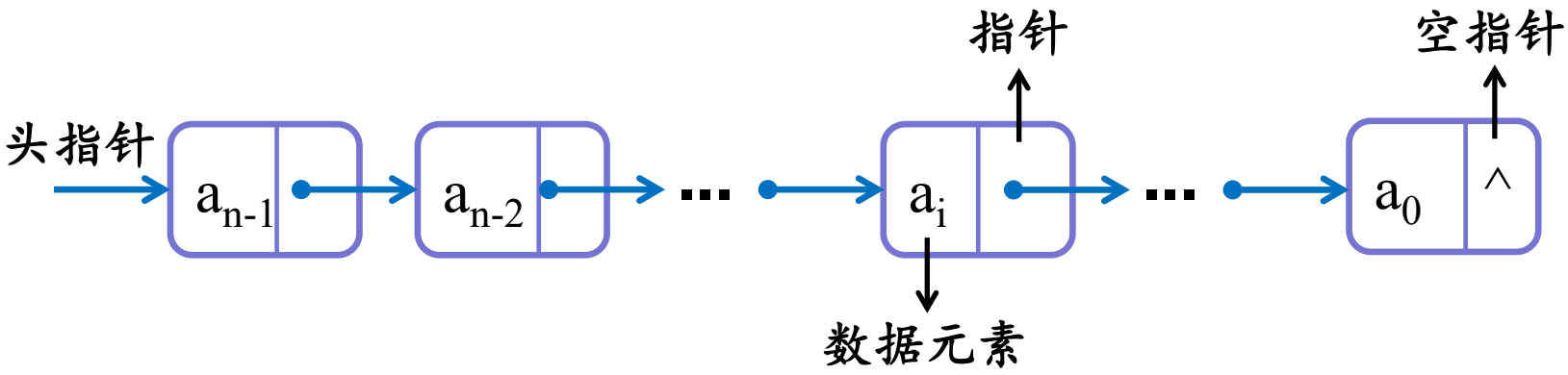
- 顺序表的特点
  - 容易实现，可以随机访问表中元素
  - 插入/删除操作涉及大量元素移动，复杂度高
  - 需连续空间存放，长度变化时可能浪费空间

本节内容

- 线性表的定义及ADT
- 顺序表
- 单向链表
- 双向链表

单向链表

- **线性表的链式存储方式-线性链表(linked list)**
  - 存放数据元素存储单元可以不连续
  - 采用额外的存储空间存放数据元素的逻辑位置
  - 通常利用附加的指针(指向前驱或者后继)，将逻辑相邻的元素连接起来



单向链表

- **单向链表**
  - 每个链表结点由两个域组成；
  - 数据域：存储数据元素；
  - 指针域：存储指向后继结点的指针.

利用友元类的方式
仅允许链表类访问
可以访问类的私有成员

data	*next

```
template <class ElemType>
class Node{
    public:
        ElemType data;
        Node<ElemType> * next;
```

问题?

```
class ClassName {

  // 成员变量和成员函数

  friend returnType functionName(arguments);
};
```

单向链

- C++实

```
template <class ElemType>
class LinkList{
    private:
        Node<ElemType>* head;
    public:
        LinkList() { head=NULL; }
        ~LinkList() { ClearList(); };
        bool ListEmpty() { return head==NULL; }
        void ClearList();
        int ListLength();
        bool GetElem(int pos, ElemType &e);
        bool SetElem(int pos, ElemType e);
        bool FindElem(int &pos, ElemType e);
        bool PriorElem(ElemType cur_e, ElemType &pre_e);
        bool NextElem(ElemType cur_e, ElemType &suc_e);
        bool Insert(int pos, ElemType e);
        bool Delete(int pos, ElemType &e);
        void Traverse(void (*visit)(ElemType &e));
}
```

单向链表

```
template <class ElemType>          //返回链表的长度
int LinkList<ElemType>::ListLength(){
    Node* p=head; int len=0;
    while(p){ p=p->next; len++; }     //空指针标明链表结尾
    return len;
}


template <class ElemType>          //返回链表第pos个元素
bool LinkList<ElemType>::GetElem(int pos, ElemType &e){
    Node* p=head; int idx=0;
    while(p && idx<pos){ p=p->next; idx++; }
    if(p==NULL)
        return false;    //pos超过链表长度
    e = p->data;
        return true;
}
```

单向链表

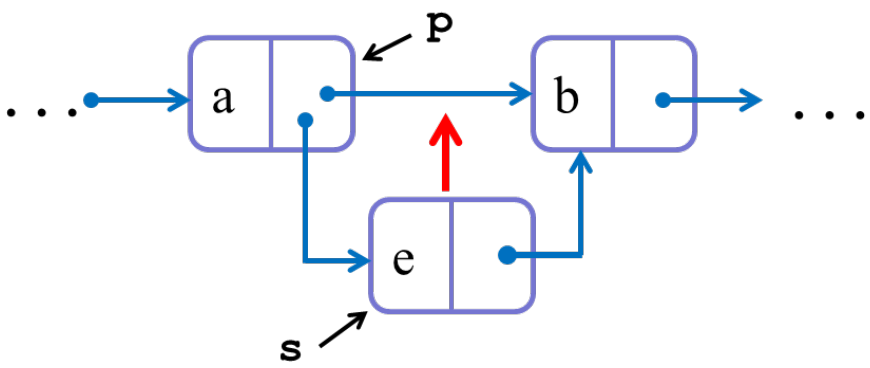
```
template <class ElemType>          //设置pos位置元素值
bool LinkList<ElemType>::SetElem(int pos, ElemType e){
    Node* p=head; int idx=0;
    while(p && idx<pos){ p=p->next; idx++; }
    if(p==NULL)     return false;    //pos超过链表长度
    p->data = e;    return true;
}


template <class ElemType>          //查找值为e的首个数据元素
bool LinkList<ElemType>::FindElem(int &pos, ElemType e){
    Node* p=head; pos=0;
    while(p && p->data!=e) { p=p->next; pos++; }
    if(p==NULL)
        return false;    //没有找到e
    else
        return true;
}
```

单

```
template <class ElemType>          //当前元素的前驱
bool LinkList<ElemType>::PriorElem(ElemType cur_e,
ElemType &pre_e){
    int pos;
    if(FindElem(pos, cur_e)){
        if(pos>0){                    //not the first elem
            GetElem(pos-1,pre_e);
            return true; }
    }
    return false;
}

template <class ElemType>        //当前元素的后继
bool LinkList<ElemType>::NextElem(ElemType cur_e, ElemType
&pre_e){
    int pos;
    if(FindElem(pos, cur_e)){
        if(pos< ListLength()-1){     //not the last elem
            GetElem(pos+1,pre_e);
            return true; }
    }
    return false;
}
```

单向链表

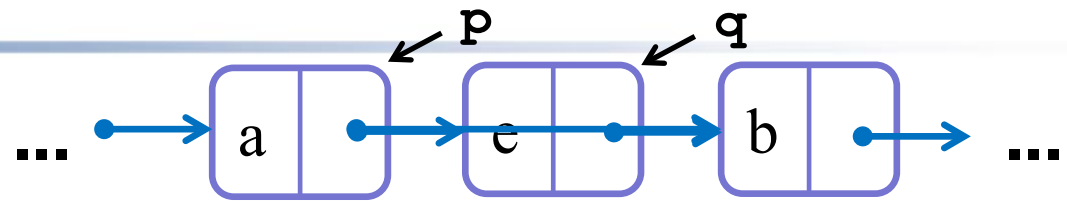
- **在链表中插入新结点**




```
template <class ElemType>        //在pos位置插入新数据元素
bool LinkList<ElemType>::Insert(int pos, ElemType e){
    if(pos<0||pos>ListLength()) return false; //超出范围
    Node *s=new Node(); s->data=e;
    if(pos==0)
    { s->next=head; head=s; return true;} //插入头位置
    Node* p=head; int idx=0;
    while(idx<pos-1){ p=p->next; idx++; } //插入位置
    s->next = p->next;    p->next = s;
    return true;
}
```

单向链表

- **在链表中删除结点**




```
template <class ElemType>        //删除pos位置的数据元素
bool LinkList<ElemType>::Delete(int pos, ElemType &e){
    if(pos<0||pos>ListLength()-1) return false; //超出范围
    Node *p = head, *q;
    int idx = 0;
    if(pos==0) {
        e = head->data; head = head->next;
        delete p; p=NULL; return true;      //删除头元素
    }
    while(idx<pos-1){ p=p->next; idx++; } //寻找删除位置
    q = p->next; p->next = q->next;
    e=q->data; delete q; q=NULL;      //取出元素并销毁结点
    return true;
}
```

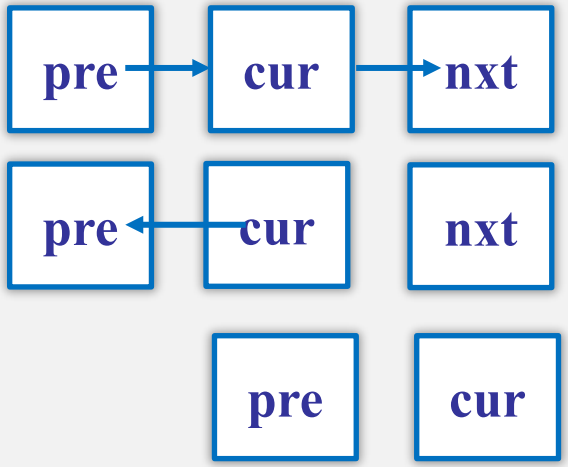
```
template <class ElemType>      //遍历表中全部数据元素
bool LinkList<ElemType>::Traverse(bool
(*visit)(ElemType & e)){
    Node *p = head,
    while( p && visit(p->data)) p=p->next;
    return p==NULL;            //是否遍历所有元素
}
```

```
template <class ElemType>      //清空链表
LinkList<ElemType>::ClearList(){
    Node *p = head, *q;
    while(p){
        q = p; p = p->next;
        delete(q); q=NULL;     //逐个释放结点
    }
    head = NULL;
}
```

练习：用非递归的方式将链表逆序

单向链表

```
Node * reverseLinkList(Node * head) {
if (head == NULL || head->next==NULL) return head;

Node *pre = head;
Node *cur = pre->next;
Node *nxt=NULL;
while (cur != NULL) {
        nxt = cur->next;
        cur->next = pre ;
        pre = cur;
        cur = nxt;
}
head->next = NULL;
return pre;
}
```



单向链表

- **求以线性表la，lb表示的两个集合的差集.**

```
void diff(LinearList la, LinearList lb, LinearList &lc){
    ClearList(lc);                          //清空lc表
    for(i=0;i<ListLength(la);i++){
        GetElem(la, i, e);                  //取出la表中元素
        if(FindElem(lb,e,equal)==-1){       //查找lb表
            Insert(lc,ListLength(lc),e);    //插入操作
        }
    }
}                                           //lc=la-lb
```

线性表（P10）

- **求以线性表la，lb表示的两个集合的差集.** 顺序表 SeqList

```
template <class ElemType>
void diff(SeqList<ElemType> la, SeqList<ElemType> lb,
SeqList<ElemType> &lc){

    ElemType e;  int pos;
    lc.ClearList();                          //清空lc表
    for(i=0;i<la.ListLength();i++){
        la.GetElem(i, e);                   //取出la表中元素
        if(!lb.FindElem(pos,e)){             //查找lb表
            lc.Insert(ListLength(lc),e); //插入操作
        }
    }
}                                            //lc=la-lb
```

单向链表

- 求以线性表la，lb表示的两个集合的差集. 单链表 LinkList

```
template <class ElemType>
void diff(LinkList<ElemType> la, LinkList<ElemType> lb,
LinkList<ElemType> &lc){

    ElemType e;  int pos;
    lc.ClearList();                          //清空lc表
    for(i=0;i<la.ListLength();i++){
        la.GetElem(i, e);                   //取出la表中元素
        if(!lb.FindElem(pos,e)){             //查找lb表
            lc.Insert(ListLength(lc),e); //插入操作
        }
    }
}                                            //lc=la-lb
```

- 封装->程序复用！
- 完全的复用？

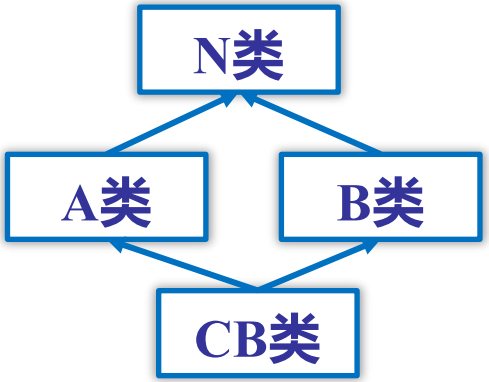
单向链表

- **定义基类LinearList，SeqList和LinkList均继承自LinearList，并实现成员函数.**

```
Class LinearList {…}

Class SeqList:virtual public LinearList {…}

Class LinkList:virtual pulic LinearList {…}
```

虚继承

```
class N
  { public:
    int a;
    void display( ) { cout << A::a = " << a << endl; }
  };
class A:public N
  { public:
    int a1;
  };
class B:public N
  { public:
```



虚基类并不是在声明基类时声明的，而是在声明派生类时，指定继承方式时声明的，因为一个基类可以在生成一个派生类时作为虚基类，而在生成另一个派生类时不作为虚基类

Class 派生类名：virtual 继承方式 基类名

即在声明派生类时，将关键字virual加到相应的继承方式前面。经过这样声明后，当基类通过多条派生路径被一个派生类继承时，该派生类只继承该基类一次，也就是说，基类成员只保留一次。

C++按值传递方式的问题

```
template <class ElemType>
void diff(LinearList<ElemType> la, LinearList<ElemType>
lb, LinearList<ElemType> &lc){

    ElemType e;  int pos;
    lc.ClearList();                          //清空lc表
    for(i=0;i<la.ListLength();i++){
        la.GetElem(i, e);                   //取出la表中元素
        if(!lb.FindElem(pos,e)){             //查找lb表
            lc.Insert(ListLength(lc),e); //插入操作
        }
    }
}                                            //lc=la-lb
```

Slicing problem

Slicing Problem 切割问题

```cpp
#include <iostream>

class Base {
    public:
        virtual void foo() {
            std::cout << "Base foo" << std::endl;
        }
};
class Derived : public Base {
    public:
        virtual void foo() {
            std::cout << "Derived foo" << std::endl;
        }
};

void func(Base b) {
    b.foo();
}

int main() {
    Derived d;
    func(d);
    return 0;
}
```

```
Base foo
```

单向链表

```
void diff(LinearList<ElemType> la,
LinearList<ElemType> lb, LinearList<ElemType> &lc);
```

```
LinkList<int> la, lb, lc;
…
diff(la, lb, lc);
```

问题? (Slicing problem)

```
void diff(LinearList<ElemType> &la,
LinearList<ElemType> &lb, LinearList<ElemType> &lc);
```

单向链表

- **单向链表的插入/删除等操作常需单独处理头指针；**
- **带表头结点的单向链表：**
  - 用一个额外的表头结点替代头指针；
  - 头结点的数据元素可以不在数据集合中；
  - 头结点在链表为空时依然存在；
  - 统一处理表中各个位置的插入删除等；

```
template <class ElemType>
class LinkList{
    private:
        Node head;
    public:
        LinkList() {
            head=Node();  head->next=NULL;
        }
        ...
}
```

单向链表

- **单向链表的特点:**
  - 有效利用存储空间，灵活改变长度(合并/分解);
  - 插入/删除操作无需移动大量数据元素;
  - 数据元素之间的顺序关系被隐式的包含;
  - 表长需要通过遍历才能获取;
  - 插入/删除操作本身很方便，但寻找相应的位置复杂度较高.

- **可能的改进方式:**
  - 增加如‘表长’，‘表尾指针’以及‘当前位置指针’等数据域
  - 基本操作中的位置’pos’可以由指针替代

单向链表

顺序表 *vs* 单向链表

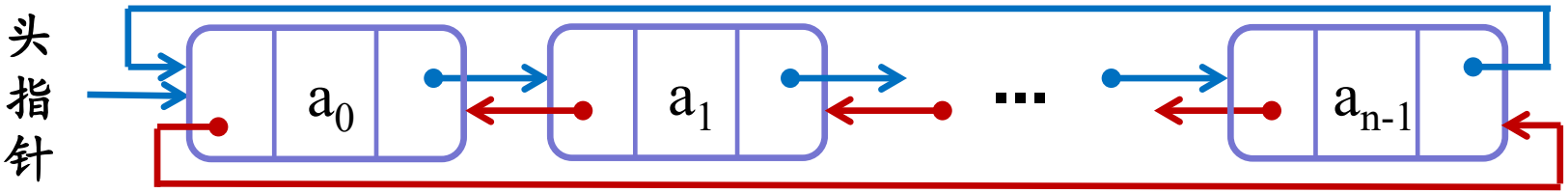
基本操作	功能	顺序表	单向链表
InitList()	构造	O(1)	O(1)
DestroyList()	析构	O(1)	O(n)
ListEmpty()	判空	O(1)	O(1)
ListLength()	表长	O(1)	O(n)
GetElem()	取值	O(1)	O(n)
SetElem()	赋值	O(1)	O(n)
PriorElem()	前驱	O(n)	O(n)
NextElem()	后继	O(n)	O(n)
FindElem()	定位	O(n)	O(n)
Insert()	插入	O(n)	O(n)
Delete()	删除	O(n)	O(n)
Traverse()	遍历	O(n)	O(n)
ClearList()	清空	O(1)	O(n)

本节内容

- 线性表的定义及ADT
- 顺序表
- 单向链表
- 双向链表

双向链表

- **增加了‘当前位置指针’的单向链表：**
  - NextElem()容易实现，复杂度O(1)
  - PriorElem()较困难，复杂度O(n)
- **可在每个节点增加指向前驱的指针，构成双向链表(doubly linked list)**

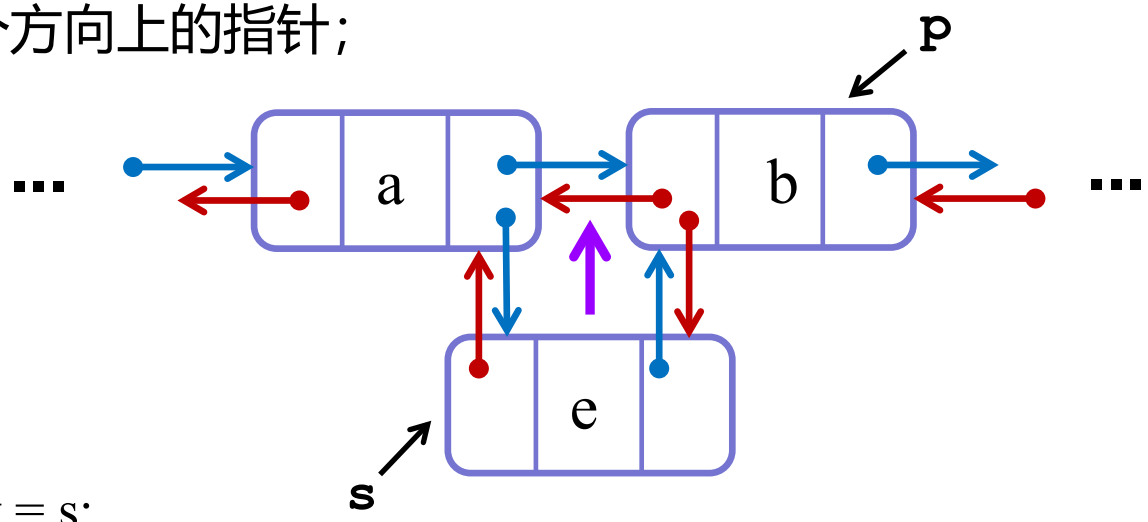


链表结点类

```
template <class ElemType>
class DNode{
    friend class DlkList;
  private:
    ElemType data;
    DNode<ElemType> * next;
    DNode<ElemType> * prev;
}
```

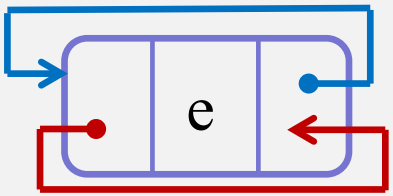
双向链表

- **双向链表的插入操作**
  - 需要修改两个方向上的指针；



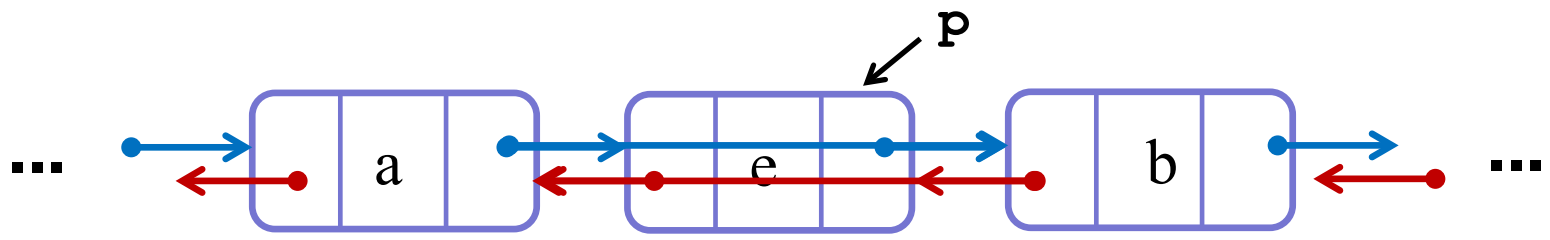
  - P->prev->next = s;
  - s->next = p;
  - s->prev = p->prev;
  - p->prev = s;

```
template <class ElemType>        //在pos位置插入新数据元素
bool DlkList<ElemType>::Insert(int pos, ElemType e){

    DNode *p=head; *s;
    if(!p){                      //链表为空
        s = new DNode();         //创建单结点链表
        s->data = e; s->prev = s;
        s->next = s; head = s;
        return true;
    }

    if(!(p=GetElemPtr(pos))) return false; //找不到位置
    s = new DNode();
    s->data = e;
    s->prev = p->prev; p->prev->next = s;  //插入操作
    s->next = p; p->prev = s;
    return true;

}
```



双向链表

- **双向链表的删除操作**



- p->prev->next = p->next;
- p->next->prev = p->prev;
- delete p;

```
template <class ElemType>                    //在pos位置插入新数据元素
bool DlkList<ElemType>::Delete(int pos, ElemType &e){

    DNode *p=head;
    if(!p) return false;                     //链表为空，无法删除
    if(!(p=GetElemPtr(pos))) return false; //找不到位置
    e = p->data;
    p->prev->next = p->next;
    p->next->prev = p->prev;
    if(p==p->next) head = NULL;    //单结点链表
    if(p==head)    head = p->next;//删除的为头结点
    delete p; p=NULL;
    return true;

}
```

双向链表

- **线性表的顺序存储-顺序表**
  - 用连续的存储单元依次存储数据元素，数据的物理关系与逻辑关系一致；
  - 优势：支持随机存储，某些操作时间复杂度较低；实现较为直接；
  - 问题：插入/删除需移动大量数据元素；存储空间利用率不高；表长变化不够灵活.
- **线性表的链式存储-链表**
  - 用物理上‘离散’的存储单元附加指针，存储线性表的数据元素；
  - 优势：存储空间利用灵活；表长易变化；插入/删除无需移动其他数据元素；
  - 问题：不支持随机存储；很多操作涉及遍历整个表；指针存储有一定开销，指针操作容易出现漏洞.

小结

- 抽象数据类型可以依托不同的物理实现
- 不同的物理实现对性能有不同的影响
- 选择合理的实现能够改善程序效率
- 在C++中，已经提供了基于数组和链表的线性表实现，在STL库中，即：
  - std::vector
  - std::forward_list
  - std::list

实验一

- 设有一个线性单链表，其节点值均为正整数，且按值从大到小链接。试写出一个算法，将该线性单链表分解为两个线性单链表，其中一个链表中的节点值均为奇数，且从小到大链接；而另一个链表中的节点均为偶数，且从小到达链接。

谢　谢！

北京科技大学
计算机与通信工程学院
杜　冰